

Article

Few-Shot Open-Set Ransomware Detection Through Meta-Learning and Energy-Based Modeling

Yun-Yi Fan ¹, Cheng-Yu Chiang ¹ and Jung-San Lee ^{1,2,*} 

¹ Department of Information Engineering and Computer Science, Feng Chia University, Taichung 407102, Taiwan; a19990101152@gmail.com (Y.-Y.F.); chengyuijiang36@gmail.com (C.-Y.C.)

² Cybersecurity Technology Institute, Institute for Information Industry, Taipei 106214, Taiwan

* Correspondence: leejs@fcu.edu.tw

Abstract

As network communication technologies rapidly advance, ransomware has emerged as a significant cybersecurity threat that organizations cannot ignore. Static analysis enables rapid identification of ransomware by examining file structure and code characteristics before execution. However, existing classifiers are predominantly designed under the closed-set assumption, causing them to misclassify novel variants into known families. Furthermore, ransomware datasets typically exhibit long-tailed distributions with emerging families having very few available samples, making it difficult for models to learn discriminative features. To address these challenges, we propose Few-Shot Open-Set Ransomware Detection through Meta-learning and Energy-based Modeling (MEM), a unified open-set recognition framework based on static analysis of Portable Executable features. By integrating Model-agnostic Meta-learning (MAML), the model rapidly adapts to new families with limited samples. The Energy Function quantifies the confidence of predictions in distinguishing between known samples and unknown ones, while Focal Loss dynamically adjusts sample weights to reduce bias introduced by imbalanced distributions. The experimental results demonstrate that MEM achieves higher classification accuracy and better rejection performance of unknown samples than existing open-set recognition methods.

Keywords: ransomware detection; open-set recognition; meta-learning; energy-based model; few-shot learning; static analysis; portable executable; long-tailed distribution

1. Introduction

As network communication technologies continue to advance, organizations are increasingly confronted with escalating information security challenges. Among these challenges, ransomware poses a significant threat that organizations cannot ignore. According to the latest cybersecurity threat intelligence report from Cyble [1], the number of global ransomware attacks has increased year by year, as illustrated in Figure 1. In January 2025, the attack volume more than tripled compared to the same period in previous years. This trend highlights the critical importance of effective ransomware prevention. Training detection models on known ransomware samples improves classification accuracy. However, performance typically degrades when encountering ransomware variants, primarily due to the limited availability of representative training data. Nevertheless, ransomware variants are not entirely independent, since most originate from a small number of major ransomware families and preserve core behavioral and structural characteristics. Therefore,



Academic Editor: Douglas O'Shaughnessy

Received: 31 January 2026

Revised: 22 February 2026

Accepted: 26 February 2026

Published: 28 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

these variants can be classified as part of the same ransomware family. As a result, detection techniques that can identify unknown threats are essential to address the insufficient protection against variant ransomware.

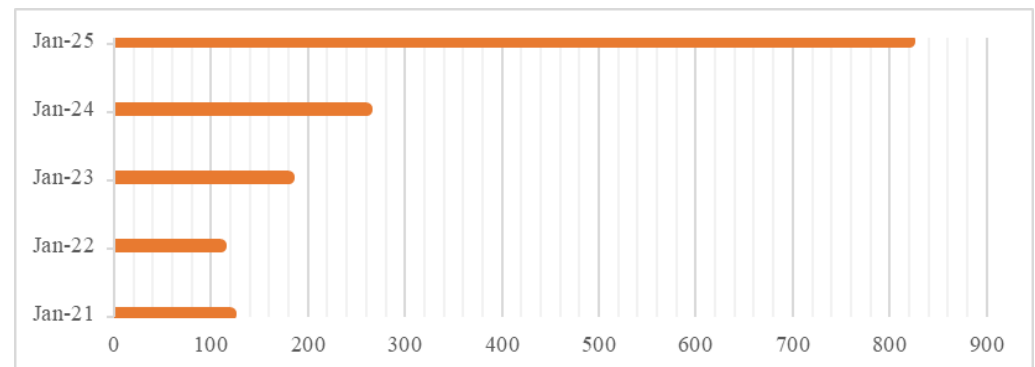


Figure 1. Statistics of Ransomware Attacks from 2021 to 2025.

Ransomware detection typically relies on monitoring network traffic or identifying characteristic features. Traditionally, signature-based detection [2] compares unknown software against a database of signature patterns to determine if it matches any known ransomware. However, signature-based detection can only identify signature patterns already recorded in the database, making it difficult to detect or block new ransomware variants. As a result, anomaly detection techniques are frequently used to detect unknown ransomware, primarily through dynamic and static analysis. Dynamic analysis involves executing unknown programs in a simulated environment to observe whether they exhibit abnormal or malicious behavior. However, dynamic analysis typically requires extended monitoring time, making real-time detection difficult. Additionally, malware can employ various anti-analysis techniques to evade monitoring, thereby reducing the effectiveness of dynamic analysis [3]. In contrast, static analysis does not require program execution and can directly examine malware's file structure and code characteristics, enabling rapid detection before ransomware is executed.

As ransomware continues to evolve, training datasets struggle to capture all emerging families. Traditional classifiers are typically designed under the closed-set assumption, limiting their ability to accurately identify categories present only during training. Consequently, when encountering novel ransomware samples, these classifiers often misclassify them into the most similar known families. To enable models to recognize unknown samples, Open-Set Recognition (OSR) methods have recently become an important research focus. However, OSR-based static ransomware analysis faces three major challenges. First, ransomware datasets often exhibit a pronounced long-tailed distribution, leading models to overfit to families with abundant samples while neglecting those with scarce samples, resulting in class imbalance bias [4]. Second, newly emerging ransomware families have limited samples in their early stages, which makes it difficult for models to effectively extract discriminative features from scarce training data. This challenge is known as the few-shot learning problem [5]. Third, even if the issues related to long-tailed distributions and few-shot learning are mitigated, models must still be able to recognize unknown samples through out-of-distribution (OOD) detection [6], allowing them to label unseen samples as unknown rather than erroneously assigning them [7]. As Moreira et al. [8] noted, if training data fails to sufficiently cover potential new patterns, the model's generalization ability for unknown sample recognition will be limited, thus restricting its practical applicability.

To address the aforementioned issues, several studies have proposed targeted solutions for individual challenges. For example, Dina et al. [9] utilized Focal Loss to dynam-

ically adjust sample weights, reducing the impact of easy examples on the loss function and thereby enhancing the recognition of hard negatives. Although this approach mitigates bias induced by class imbalance, it remains constrained within a closed-set framework and cannot possess the capacity to effectively identify samples from unknown classes. Zhu et al. [10] enhanced classification stability by employing similarity learning and Center Loss to strengthen inter-family feature distinctions. However, this framework lacks a decision mechanism for recognizing unknown families, often misclassifying them as known families. On the other hand, Ji et al. [11] used Model-agnostic Meta-learning (MAML) to obtain generalized initial parameters, enabling rapid convergence under conditions of limited samples. Nonetheless, this approach relies heavily on the similarity between training and testing data distributions and does not incorporate mechanisms to identify unknown samples, resulting in ineffective handling of novel families. Traditional OSR methods often rely on Extreme Value Theory (EVT) to estimate a rejection threshold, which is used to assess the probability that a sample belongs to a known class and decide whether to reject it. OpenMax is a representative example of such methods. However, under conditions of limited samples and long-tailed distributions, the fitting process for EVT often becomes inaccurate due to insufficient sample size or feature distribution shifts. This leads to unstable rejection thresholds, which in turn affect the overall reliability of recognition. Guo et al. [12] employed Generative Adversarial Networks (GANs) to synthesize unknown samples, thereby helping the model learn rejection decisions and expanding the separation boundary between known and unknown samples. However, when tail family samples are extremely scarce, GANs struggle to capture representative latent distributions. The generated samples often suffer from mode collapse and distribution shift, resulting in a lack of diversity and potentially containing features of invalid PE structures, which further weakens the ability of the model to recognize unknown samples. Lu et al. [13] proposed the DOMR framework, which uses meta-learning to simulate open-world scenarios, enhancing the model's adaptability to unknown classes. Although this method shows promise in open-set recognition experiments, its effectiveness heavily depends on sufficient training samples. The authors also note that when the sample size per family is small, the model's generalization ability significantly deteriorates.

In summary, previous research has predominantly focused on individual challenges, making it difficult to simultaneously address the demands of extreme data distributions, few-shot conditions, and unknown sample recognition. To address these limitations, this study introduces a unified framework titled Few-Shot Open-Set Ransomware Detection through Meta-learning and Energy-based Modeling (MEM), which integrates three complementary techniques: MAML, the Energy Function from Energy-based Model (EBM), and Focal Loss, each corresponding to one of the aforementioned challenges. First, MAML is employed to learn generalized initial parameters, enhancing the ability of the model to rapidly adapt to extremely limited samples. Second, the Energy Function quantifies the model's confidence in its predictions, helping establish a stable and adjustable rejection threshold. Lastly, Focal Loss dynamically adjusts the loss contribution during fine-tuning to alleviate overfitting on head classes and increase the learning participation of tail samples. The integration of these three components enables the proposed framework to effectively handle data imbalance and unknown class challenges in open-set ransomware recognition under static conditions, thereby improving the model's performance in OSR scenarios. The main contributions of this study are as follows:

- Strengthening model learning stability under extreme data distributions by addressing the complexities of long-tailed and few-shot scenarios.
- Enhancing the adaptability of models in open-set contexts to improve unknown sample recognition.

The structure of this thesis is organized as follows. Section 2 introduces static feature modeling and explains the theoretical foundations related to open-set recognition (OSR). Section 3 provides a detailed description of the proposed three-stage recognition framework and its training process. Section 4 presents the complete experimental design, the comparative methods, and the result analysis. Finally, Section 5 summarizes the main contributions of this study and discusses potential directions for future research.

2. Preliminaries

This section presents the core technical foundations underlying MEM. Section 2.1 describes the Portable Executable (PE) static features used as input, chosen because static analysis avoids the sandbox-evasion risks inherent in dynamic approaches and enables rapid, scalable screening. Section 2.2 introduces MAML, the meta-learning backbone that enables few-shot adaptation. Section 2.3 defines the Energy Function used to separate known from unknown samples. Section 2.4 presents Focal Loss for mitigating class imbalance, and Section 2.5 describes Center Loss for encouraging intra-class compactness.

2.1. Portable Executable Features

Moreira et al. [8] proposed a comprehensive multi-PE structure feature combination analysis method that systematically integrates five categories of static features extracted from PE files: PE header fields, section metadata, section entropy, imported DLL and API information, and opcode sequences. This approach constructs a unified feature representation by combining structural, behavioral, and content-based characteristics and has demonstrated significant effectiveness in distinguishing ransomware families through static analysis. The integration of these complementary feature types enables the capture of both high-level behavioral patterns and low-level implementation details, thereby improving classification accuracy across diverse ransomware variants.

In the Windows operating system environment, ransomware primarily relies on Windows PE files as the vehicle for propagation and execution. Therefore, accurately parsing the PE file structures and extracting static features is a necessary prerequisite for effective static analysis and identification of malicious software. As illustrated in Figure 2, the standard PE format consists of two major structural components: the header and the sections. The header provides metadata about the file's overall properties and internal indexing, while the sections contain the actual code, data, and other resources. The header region can be further divided into three substructures: the DOS Header, the NT Header, and the Section Header. The DOS Header, located at the beginning of the file, identifies the file type and provides a pointer to the NT Header. The NT Header follows and includes the File Header and the Optional Header. The File Header contains basic attributes such as the target platform and the number of sections. The Optional Header specifies parameters such as the program entry point, memory layout, and execution settings. Within the Optional Header, the Data Directories field lists critical information such as table addresses and library references. Among them, the Import Address Table (IAT) records the Dynamic-Link Libraries (DLLs) and Application Programming Interface (API) names required at runtime, serving as a key reference for static behavioral analysis. Finally, the Section Header defines the starting location, size, and access permissions of each section both on disk and in memory. This structural information can be used to detect anomalies in section configuration, which may indicate malicious behavior.

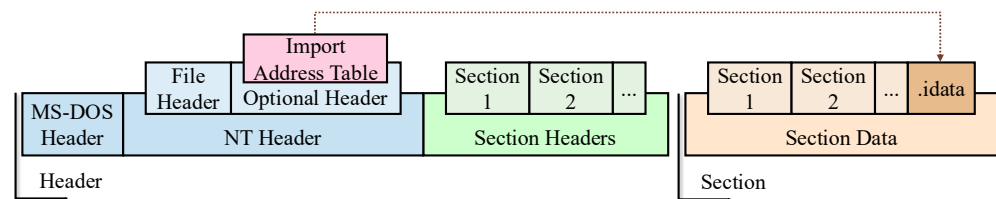


Figure 2. Main Information of the PE Structure. The dashed arrow indicates the data source of the Import Address Table.

The sections of a PE file serve as data blocks that store the program's actual content, with each section holding different types of data depending on its purpose. For example, the .text section contains executable instructions, the .data section stores global variables, and the .rsrc section includes resources such as icons and strings. These section contents can be leveraged to extract opcode sequences, compute section entropy, and examine whether any section exhibits abnormal resource configurations, which may suggest the presence of hidden malicious behavior. All of the above structural and sectional information can be considered as potential sources of static features, contributing to ransomware family classification and the detection of novel variants. Accordingly, this study systematically extracts five categories of static features based on the PE file structure, which serve as inputs to the proposed model. The following subsections provide detailed explanations of each feature type: Section 2.1.1 introduces the PE Header; Section 2.1.2 focuses on section metadata; Section 2.1.3 explains section entropy; Section 2.1.4 covers DLL and API information; and Section 2.1.5 presents opcode sequence analysis.

2.1.1. PE Header

The PE Header is located at the beginning of the file and contains essential layout information such as the program's execution environment and internal structure. As shown in Table 1, by analyzing the values of specific fields in the header, it is possible to capture the fundamental structural characteristics of a malicious executable and distinguish between different ransomware families. Studies have shown that PE header fields have a significant impact on malware classification. According to the findings of Moreira et al., the PE header is considered one of the most influential static features for ransomware detection.

Table 1. Extracted Specific PE Header Features.

Property	Specific Fields
DOS_HEADER	e_magic, e_cblp, e_cp, e_crlc, e_cparhdr, e_minalloc, e_maxalloc, e_ss, e_sp, e_csum, e_ip, e_cs, e_lfarlc, e_ovno, e_oemid, e_oeminfo, e_lfanew
FILE_HEADER	Machine, NumberOfSections, PointerToSymbolTable, NumberOfSymbols, SizeOfOptionalHeader, Characteristics
OPTIONAL_HEADER	Magic, MajorLinkerVersion, MinorLinkerVersion, SizeOfCode, SizeOfInitializedData, SizeOfUninitializedData, AddressOfEntryPoint, BaseOfCode, BaseOfData, ImageBase, SectionAlignment, FileAlignment, MajorOperatingSystemVersion, MinorOperatingSystemVersion, MajorImageVersion, MinorImageVersion, MajorSubsystemVersion, MinorSubsystemVersion, SizeOfImage, SizeOfHeaders, CheckSum, Subsystem, DllCharacteristics, SizeOfStackReserve, SizeOfStackCommit, SizeOfHeapReserve, SizeOfHeapCommit, LoaderFlags, NumberOfRvaAndSizes

2.1.2. Section Metadata

Every section is accompanied by structured metadata that defines its attributes, including its name, size, virtual address, and memory access permissions. Such metadata facilitates the reconstruction of a program's memory layout and enables comparative

analysis of section-level structures across different binaries, which can aid in identifying anomalous configurations. Irregularities in section names or sizes may suggest obfuscation techniques employed by malicious software to evade detection. Consequently, section attributes have been widely adopted as salient features in static analysis for identifying potentially suspicious or malicious samples [13,14].

2.1.3. Section Entropy

Beyond serving as a basis for understanding program structure and functionality, the content characteristics of PE file sections can also reveal potential anomalies. In the context of malware analysis, entropy-based methods are commonly employed to detect packed or encrypted portable executable files [2,9]. Sections that have been encrypted or compressed often exhibit high randomness, which can be quantitatively measured using entropy. Conversely, when a section contains highly regular content, such as repeating characters or zero-padding, it tends to have the lower entropy values, whereas more randomized content results in the higher entropy values.

Sections with abnormally high entropy may indicate that the program utilizes custom encryption or compression techniques to conceal its executable code, thereby evading signature-based detection by antivirus software. As such, entropy analysis has become a valuable auxiliary technique in static analysis for identifying suspicious section configurations and facilitating the preliminary screening of potentially malicious samples.

2.1.4. Dynamic-Link Library and Application Programming Interface

The Import Table in a PE file lists the external DLLs and corresponding API functions that must be loaded at runtime. By examining the DLLs and system APIs imported by a given binary, it is possible to infer the likely behavioral intentions of a malware sample—such as file manipulation, network communication, process injection, or registry modification.

Since the list of imported DLLs and APIs reflects the malware's dependency on specific operating system functionalities, it is often regarded as a static feature indicative of behavioral intent. Numerous studies have demonstrated that leveraging such import-related features for malware classification can significantly enhance detection accuracy [13,15].

2.1.5. Opcode Sequence

An opcode sequence is derived by disassembling the machine code in the executable sections of a binary, resulting in a series of low-level instructions that correspond to specific processor operations. Malware samples belonging to the same family often exhibit similar patterns or recurring fragments in their opcode usage. As a result, opcode sequence features can help determine whether an unknown sample is related to a known malware family.

Some studies [13,16,17] have proposed using opcode frequency patterns to detect variants of known malware families, demonstrating that such approaches are effective in identifying previously unseen malicious samples. Therefore, opcode sequences not only serve as a useful indicator of family-level similarity in static analysis but are also frequently used to construct behavioral models to improve the detection of malware variants.

2.2. Model-Agnostic Meta-Learning

MAML [18] is a meta-learning strategy that aims to find a set of highly transferable initial model weights, enabling the model to quickly adapt and achieve good performance when facing new tasks with only a small number of samples and gradient update steps.

MAML employs a two-layer optimization architecture consisting of an inner loop and an outer loop, as illustrated in Figure 3. In each training iteration, the algorithm randomly selects several classification tasks from a task pool, such as identifying different

malware families. For each task T , the training data is divided into two subsets: a support set S for task-specific parameter tuning and a query set Q for evaluating the tuned performance. In the inner loop, the model uses a small number of labeled samples from the support set to perform several gradient descent iterations starting from the initial weights to obtain task-specific temporary parameters. Then, in the outer loop, Q is used for outer layer updates. The algorithm calculates the loss based on the model's performance on the query set and backpropagates and adjusts the initial weights, making the model more adaptable to new tasks in the next iteration.

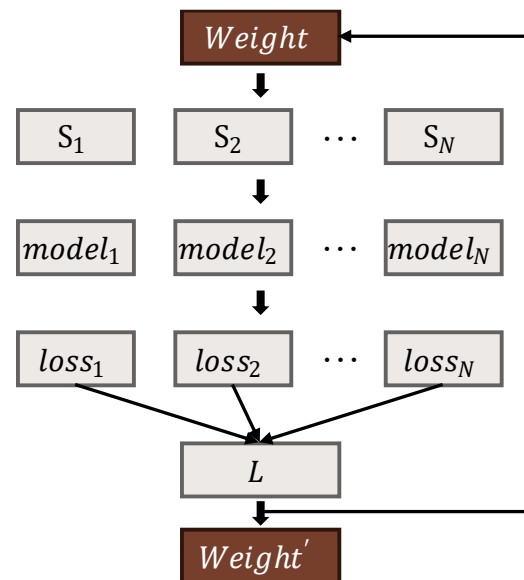


Figure 3. Architecture of MAML.

Through a two-stage training mechanism of rapid inner-layer adjustment and robust outer-layer optimization, MAML can maintain the initial parameters at a highly adaptive initial position and demonstrate strong transferability and learning stability in multi-task learning and very few-shot scenarios.

2.3. Energy Function

The Energy Function [19] is a quantitative metric that measures a classification model's confidence in an input sample. This method computes a scalar value, referred to as the Energy Value, by directly operating on the logits vector of the model output without applying Softmax normalization. The resulting value reflects the overall activation magnitude of the model output and serves as a proxy for the model's confidence.

For an input sample x , the Energy Function is defined as Equation (1):

$$E(x) = -\log \sum_C \exp(z_c), \quad (1)$$

where z_c denotes the logit output corresponding to class c , and C is the total number of classes. This formulation computes the negative log-sum-exp of the logits, capturing the model's overall response strength. When the model is highly confident in a particular class, the corresponding logit dominates the summation, resulting in a lower energy value, which indicates a clear prediction tendency. Conversely, when the logits are distributed more evenly across classes, the energy value increases, suggesting lower confidence in the prediction.

Previous studies have demonstrated that energy values can effectively distinguish in-distribution samples from out-of-distribution inputs, and can serve as a complementary mechanism for separating known and unknown samples.

2.4. Focal Loss

Focal Loss [20] is a loss function specifically designed to address class imbalance, derived from Cross-entropy Loss. In practical applications of ransomware detection, the sample sizes of different families vary significantly, resulting in a typical long-tail distribution. Traditional Cross-entropy Loss is easily dominated by the majority classes in this context, leading to poor learning performance for the minority classes. Cross-entropy Loss is the standard loss function for supervised classification tasks, measuring the difference between the model's predicted probability and the true label. For a single sample, its Cross-entropy Loss is defined as shown in Equation (2), where C is the total number of classes, y_c is the true label, and p_c is the model's predicted probability for the c -th class.

$$L_{CE} = - \sum_{c=1}^C y_c \cdot \log(p_c) \quad (2)$$

Focal Loss introduces a class balance factor and a focus parameter on top of Cross-entropy Loss, enabling the model to handle imbalanced data more effectively. Its definition is shown in Equation (3), where $\alpha_c \in [0, 1]$ is the class balance factor, which adjusts the weights based on the sample size of different classes. When the number of samples in a certain category is sparse, α_c can be set to a higher value to strengthen the loss impact of that category. γ is the focusing parameter, which controls the degree to which the model pays attention to difficult samples. When the model's prediction confidence p_c for a sample is close to 1, the $(1 - p_c)^\gamma$ term will significantly reduce the loss contribution of that sample, causing the model to focus its attention on samples that are difficult to classify correctly.

$$L_{FL} = - \sum_{c=1}^C \alpha_c \cdot y_c \cdot (1 - p_c)^\gamma \cdot \log(p_c) \quad (3)$$

2.5. Center Loss

Center Loss [21] is an auxiliary loss function specifically designed to address the feature dispersion problem in deep learning. In neural networks trained using only the Softmax loss function, the feature representations of samples within the same class are often too dispersed, which reduces the model's classification accuracy and stability.

The core idea of Center Loss is to encourage samples of the same class to cluster in the feature space. Specifically, this method maintains a dynamically updated center point for each class and minimizes the distance between all samples of that class and its center. This mechanism effectively reduces intra-class distance, making the feature distribution of the same class more compact. For the detection sample set $D = \{x_1, x_2, \dots, x_N\}$, the mathematical definition of Center Loss is shown in Equation (4), where x_i represents the feature vector of the i -th sample, and $ctr_{y_i} \in \mathbb{R}^d$ represents the center vector of the class c to which the sample belongs. During training, the class center is dynamically updated based on the latest sample features of that class.

$$L_{ctr} = \frac{1}{2} \sum_{i=1}^N \|x_i - ctr_c\|_2^2 \quad (4)$$

By minimizing the intra-class distance, the feature representations of the same class become more concentrated, and the boundaries between different classes become clearer. This

not only improves the accuracy of classification but also enhances the stability and reliability of the model when faced with new samples.

3. Proposed Method

This section introduces the proposed open-set ransomware detection framework, referred to as Few-Shot Open-Set Ransomware Detection through Meta-learning and Energy-based Modeling (MEM). As shown in Figure 4, MEM operates in three stages: data preprocessing, meta-learning optimization, and fine-tuning with energy threshold construction. Before detailing each stage, Table 2 summarizes the key symbols and notations used throughout this paper.

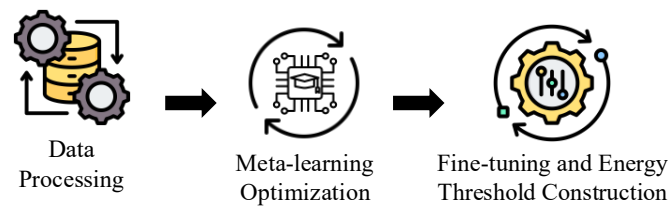


Figure 4. Architecture of MEM.

Table 2. Notation Table.

Symbol	Definition
N	Number of families randomly sampled per meta-training task, set to 20
$N - 1$	Known families within each task
1	Simulated unknown family within each task
K	Samples per family in support and known query sets, set to 5
M	Number of tasks per meta-iteration (meta-batch size), set to 64
S_j	Support set of task T_j , containing $(N - 1) \times K$ labeled samples
Q_j^k	Known query set of task T_j , containing $(N - 1) \times K$ samples, set to 95
Q_j^u	Unknown query set of task T_j , set to 95
$weight$	Meta-learned initial parameters shared across all tasks
$weight'_j$	Task-adapted parameters after inner-loop gradient steps
α	Inner-loop learning rate, set to 0.001
η	Outer-loop learning rate, set to 0.001
λ_{ctr}	Weight of Center Loss in the inner-loop objective, set to 1
m	Center Loss margin, set to 2.0
λ_k	Weight of the known-sample energy penalty in the outer-loop objective, set to 2.0
λ_u	Weight of the unknown-sample energy penalty in the outer-loop objective, set to 0.1
λ_{ft}	Weight of Focal Loss during fine-tuning, set to 0.25
TH_c	Per-family energy rejection threshold for family c
$model_{ft}$	Fine-tuned classifier after Stage 3

3.1. Data Preprocessing

Following the multi-PE structure feature combination approach described in Section 2.1, we extracted and processed key static features in the ransomware field, converting the original malware files into standardized numerical features for subsequent machine learning models. The overall process is shown in Figure 5.

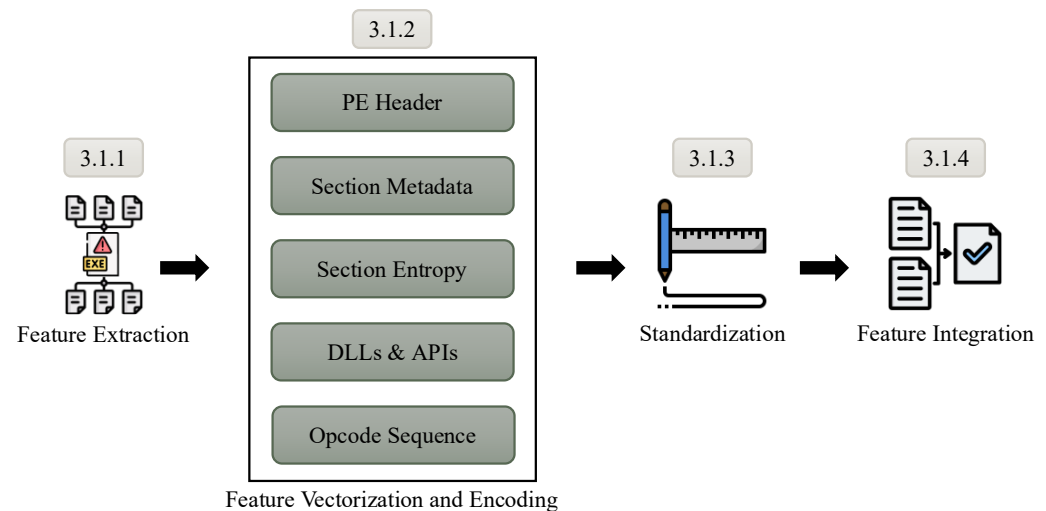


Figure 5. Static Feature Extraction and Data Preprocessing Workflow.

3.1.1. Feature Extraction

MEM reads a set of samples to be tested, $D = \{x_1, x_2, \dots, x_N\}$, and performs feature extraction on each binary file sample x_i . First, using the PE structure of x_i , it extracts the 52 field values from the PE Header as shown in Section 2.1. Next, it extracts all DLL names and corresponding API function call names from the import table, compiling a complete list of DLLs and APIs used by the sample. Then, it records the metadata information of all sections defined in the file, such as section name, size, virtual address, and permissions, and obtains the raw bit content of each section for subsequent entropy calculation. Finally, it decompiles the executable code sections of the sample, obtaining the machine instruction sequence and converting it into an opcode sequence. Through the above procedures, each sample x_i obtains a set of raw feature data covering file structure, section content, library function calls, and other information.

3.1.2. Feature Vectorization and Encoding

To ensure heterogeneous raw features from various sources can be uniformly converted into fixed-dimensional numerical vectors for subsequent model training and feature combination, this section performs transformation and vectorization processing on all feature categories extracted in the previous section. For each sample x_i , MEM processes its PE Header, Section Metadata, Section Entropy, DLL, and API names, using Opcode Sequence, and converts the above six types of information into numerical feature vectors, denoted as $header_i$, $section_i$, $entropy_i$, dll_i , api_i , and $opcode_i$. The transformation methods are designed according to various feature types, including field filtering, length unification, word frequency statistics, and vector encoding, as detailed below.

A. PE Header

PE Header information of the sample x_i is converted into a decimal numerical representation. Moreira et al. [8] report 84 PE header fields in total. MEM retains 52 fields from the DOS_HEADER, FILE_HEADER, and OPTIONAL_HEADER structures and excludes the Data Directory entries. A zero-variance filter then removes any field that is constant across all training samples; in the RCFS dataset, all 52 fields pass this filter. The resulting vector is denoted as $header_i$.

B. Section Metadata

The metadata of all segments of sample x_i are converted into a decimal numerical representation. MEM then compiles a fixed-order list of segments by summarizing the

segment names appearing in all samples. For samples lacking certain segments, we fill in zeros to ensure that each sample obtains a segment feature vector of consistent length $section_i$.

C. Section Entropy

MEM first aggregates the original bit content of the segment columns of all samples x_i and then calculates the bit entropy value of each segment in sample x_i . If a sample is missing a specific segment, the corresponding entropy value is filled with 0 to ensure that the dimension of each entropy feature vector $entropy_i$ is consistent.

D. DLL and API

For the sample x_i containing a DLL and API function list, MEM unifies the names of APIs with similar functions as described in Section 2.1. Then, it applies Term Frequency-inverse Document Frequency (TF-IDF) [22] in text processing to calculate a weighted average of the DLL and API names. Each dimension of the DLL or API dictionary vector of each sample corresponds to a specific DLL or API name. If the sample imports that item, the value is $tf \times idf$ weight; otherwise, it is 0. In this way, the function libraries and function call lists, which were originally of different lengths, are transformed into numerical vectors of the same length, which serve as the DLL feature dll_i and the API feature api_i , respectively. The DLL and API vocabularies are constructed from the training set of each run; tokens appearing in fewer than two samples are discarded to suppress noise. After filtering, the DLL vocabulary and API vocabulary yield approximately 500 and 1000 dimensions respectively, though the exact count varies slightly across runs due to the randomized family selection.

E. Opcode Sequence

MEM extracts opcode fragment features of length 3-gram from each sample x_i , performs n-gram analysis, as in Section 2.1., counts the frequency of various short instruction fragments in the sequence, and builds a vocabulary of opcode fragments accordingly. Then, it converts the opcode sequence of each sample into a corresponding frequency vector $opcode_i$ according to the vocabulary, where each dimension represents the number of times or frequency of a specific opcode fragment appears in the sample.

3.1.3. Numerical Feature Standardization

It performs standard score scaling on all features, including $header_i$, $section_i$, $entropy_i$, dll_i , api_i , and $opcode_i$. Each feature dimension is transformed to have zero mean and unit standard deviation, preventing features with different scales from disproportionately influencing model training.

3.1.4. Feature Integration

For each sample x_i , its various feature vectors are concatenated sequentially, and the integrated result is denoted as $F_i = [header_i|section_i|entropy_i|dll_i|api_i|opcode_i]$, and F_i is combined with its corresponding true label y_i to form a complete training dataset.

3.2. Meta-Learning Optimization

After receiving F_i in MAML, MEM designs a two-layer optimized meta-learning optimization framework to enable the model to quickly learn ransomware identification from a small number of samples. This framework learns highly transferable initial parameters by simulating multiple few-shot learning tasks, while introducing an Energy Function-based loss mechanism to enhance the ability of the model to distinguish between known and unknown samples. As shown in Figure 6, the architecture adopts a two-layer loop structure, with the meta-learned initial parameters $weight$ as the core of the framework.

For the i -th task T_j , the inner optimization calculation L_{inner}^j starts from the support set S_j , generating the adaptive model parameters $model_j$. Then, the outer loss L_{outer}^j is calculated using the known query set Q_j^k and the unknown query set Q_j^u . The losses from all tasks are accumulated through gradients to update the initial parameters to $weight'$.

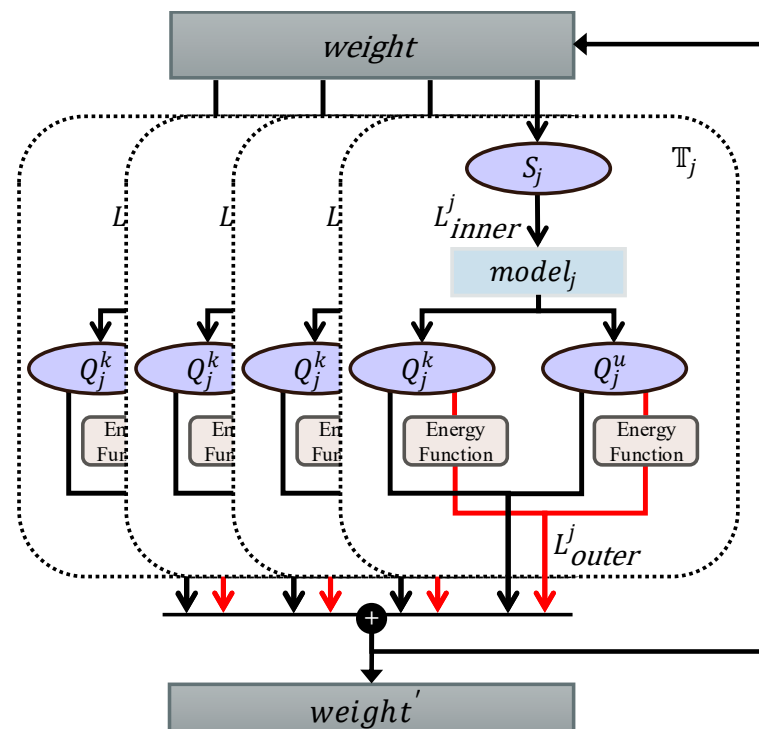


Figure 6. Meta-learning Optimization Architecture. Black arrows denote the conventional MAML data flow; red arrows indicate the Energy Function computation paths; dashed boxes enclose individual sampled tasks.

3.2.1. Task Building

The key to meta-learning lies in constructing training tasks that can simulate real-world, low-sample scenarios. In each meta-training iteration, the system randomly constructs M tasks $T_1, T_2, \dots, T_M$. Each task T_j contains three components: a support set, a known query set, and an unknown query set, to fully simulate the challenge of open-set ransomware identification. The entire task construction process follows these steps:

Step 1: For task T_j , we first randomly select N ransomware families from all available families in the training data. $(N - 1)$ families are designated as known categories, and the remaining family serves as the source of unknown samples, so that across tasks the model encounters diverse known/unknown partitions.

Step 2: S_j is constructed using a stratified sampling strategy. From the selected $(N - 1)$ known families, K samples and their corresponding labels are randomly selected from each family. The support set simulates the situation in real-world applications where only a small number of labeled samples are available in the early stages of a new ransomware outbreak, and is used for updating the model's inner parameters and for rapid adaptation.

Step 3: The construction of the query set consists of two parts. The known query set Q_j^k draws K samples from each of the $(N - 1)$ known families. If the total number of samples in a family is insufficient to support completely non-overlapping sampling, duplicate samples with the support set are allowed. The unknown query set Q_j^u randomly draws a fixed number of samples from the previously retained unknown family. The purpose of the query set design is to evaluate the model's overall performance after adaptation. The

known query set is used to evaluate classification ability, and the unknown query set is used to evaluate rejection ability.

3.2.2. Inner-Loop Optimization

The inner-loop optimization simulation model is a rapid adaptation process when encountering a new task. Its goal is to enable the model to quickly adjust its parameters to adapt to the feature distribution of a specific task based on a small number of supporting samples for that task. It consists of the following steps:

Step 1: In task T_j , the model computes the inner loss function L_{inner}^j on the support set S_j . This loss function combines two complementary components: Cross-entropy loss and Center Loss, to simultaneously ensure classification accuracy and feature distribution quality. The specific definition is shown in Equation (5). Where $L_{CE}(S_j)$ is the standard Cross-entropy loss, measuring the model's classification accuracy on the support set and ensuring the model can correctly identify known ransomware families. $L_{ctr}(S_j)$ is the Center Loss, as shown in Section 2, which strengthens the clustering of features within the same class by minimizing the Euclidean distance from samples of the same class to the class center. λ_{ctr} , as a weight hyperparameter, is used to balance the contributions of the two losses; its value directly affects the trade-off between classification performance and feature clustering.

$$L_{inner}^j = L_{CE}(S_j) + \lambda_{ctr} L_{ctr}(S_j) \quad (5)$$

Step 2: Inner layer parameter updates employ standard gradient descent. Let $weight^0$ be the common initial meta-parameters for all tasks $T_1, T_2, \dots, T_M$. In task T_j , the adapted parameter after inner layer optimization is $weight_j'$, and the update rule is shown in Equation (6). Here, α is the inner layer learning rate, and ∇_{weight} represents the gradient operation on parameter $weight$. We adopt a three-gradient update design, which provides a good balance between computational efficiency and adaptation effect, allowing the model to fully adapt to the current task while avoiding the decline in generalization ability caused by overfitting.

$$weight_j' \leftarrow weight^0 - \alpha \nabla_{weight} L_{inner}^j \quad (6)$$

3.2.3. Outer-Loop Optimization

Outer loop optimization is a core innovation of the MAML framework, responsible for updating the meta-parameter $weight$ to enable the model to adapt quickly to various tasks while distinguishing between known and unknown samples. It consists of the following steps:

Step 1: Outer Loop Optimization. First, an open-set detection mechanism based on an Energy Function is established. For an input sample x , its Energy Function $E(x)$ is defined by Equations (7) and (8). The energy values of all samples in Q_j^k and Q_j^u are calculated after being output by the model $f(weight_j')$. Based on the above characteristics of the Energy Function, two types of Energy Loss functions are defined as shown in Section 2.4, where $(N-1)$ represents the number of known families in the current task, $[z]_+ = \max(0, z)$ is the modified linear unit activation function, $|Q_j^k|$ and $|Q_j^u|$ represent the number of samples in the known and unknown query sets, respectively, and th is the energy threshold.

$$L_{E,k}^j = \frac{1}{|Q_j^k|} \sum_{x \in Q_j^k} [E(x) - th] \quad (7)$$

$$L_{E,u}^j = \frac{1}{|Q_j^u|} \sum_{x \in Q_j^u} [th - E(x)] \quad (8)$$

Observing the loss functions Q_j^k and Q_j^u , it can be found that the known sample Energy Loss $L_{E,k}^j$ only produces a positive penalty when the energy value $E(x)$ of the known sample exceeds the threshold th , forcing the model to reduce the energy value of the known sample to improve the confidence level of the judgment. The unknown sample Energy Loss $L_{E,u}^j$ has the opposite mechanism, only producing a positive penalty when the energy value $E(x)$ of the unknown sample is lower than the threshold th , prompting the model to increase the energy value of the unknown sample to reduce the risk of misclassification. MEM uses a fixed threshold $th = -\log(N - 1)$. According to the definition of the Energy Function, when all outputs are zero, the energy value is approximately $-\log(N - 1)$, where $(N - 1)$ is the number of known categories of task T_j . Through extensive experimental observation, it was found that the known and unknown sample energy values are most separable around this threshold.

Step 2: The outer loop uses the query set $Q_j = Q_j^k \cup Q_j^u$ to evaluate the overall performance of the model $f(weight'_j)$ after adaptation by the inner loop, and calculates the outer loss function containing three important components. As shown in Equation (9), this loss function adopts a multi-objective optimization design. The first term, $L_{CE}(Q_j^k)$, is the classification loss for known samples, ensuring that the model maintains good classification performance for known family samples after fine-tuning with the support set, preventing misclassification of known families during the adaptation process. The last two terms, $L_{E,k}^j$ and $L_{E,u}^j$, are loss terms designed based on the Energy Function, specifically used to train the model to learn the key ability to distinguish between known and unknown distribution samples.

The weight hyperparameters λ_k and λ_u are used to adjust the relative importance among the three loss terms. λ_k controls the strength of the energy constraint for known samples, and λ_u controls the strength of the energy constraint for unknown samples. The appropriate setting of these two weight hyperparameters has a decisive impact on balancing classification accuracy and open-set detection performance.

$$L_{outer}^j = L_{CE}(Q_j^k) + \lambda_k L_{E,k}^j + \lambda_u L_{E,u}^j \quad (9)$$

Step 3: The initial meta-learning parameters are updated by averaging the outer-layer loss across all M tasks and performing gradient descent, as shown in Equation (10). Here, L_{outer}^j is the outer layer learning rate. Since the outer layer loss L_{outer}^j depends on the model parameters $weight'$ updated via the inner loop, as calculated in Equation (9), this mechanism allows MAML to learn a good initial parameter position. From this position, it can quickly adapt to new tasks with only a few gradient steps, achieving the core goal of meta-learning. Through iterative execution of task construction, inner loop adaptation, and outer loop update, the initial parameters $weight$ of meta-learning gradually converge to the optimal configuration, ultimately yielding initial parameters that combine classification and open-set detection capabilities.

$$weight \leftarrow weight - \eta \nabla_{weight} \left(\frac{1}{M} \sum_{j=1}^M L_{outer}^j(weight') \right) \quad (10)$$

3.3. Fine-Tuning and Energy Threshold Construction

After the aforementioned meta-learning stage, we obtained a general meta-learning initial parameter $weight$ with rapid adaptability. When encountering a specific ransomware family that has not been previously seen during actual deployment, we need to fine-tune the model and set an energy threshold to further optimize the model for that family, while

establishing a decision-making mechanism to distinguish between known and unknown samples. The steps are as follows:

Step 1: We use the detection sample set D to train and fine-tune the meta-learning initial parameter *weight*, and use a weighted combination of Cross-entropy loss and Focal Loss L_{FL} as the loss function for fine-tuning to reduce the impact of data imbalance. The final fine-tuned model $model_{ft}$ is shown in Equation (11).

$$L_{ft} = (1 - \lambda_{ft}) L_{CE} + \lambda_{ft} L_{FL}, \quad (11)$$

where L_{CE} is the traditional Cross-entropy loss, L_{FL} is the Focal Loss, and λ_{ft} is a hyperparameter that balances the ratio of the two.

Step 2: MEM uses $model_{ft}$ to calculate the energy scores $E(x)$ of all target family c in the training set. These $E(x)$ represent the model familiarity with the samples of that family. Next, we calculate the mean and standard deviation of these energy scores, denoted as μ_c and σ_c , respectively. As defined in Equation (12), this threshold is used to filter low-energy samples, serving as the dividing line between this family and unknown samples. It also avoids the influence of extreme values, improving the stability of rejection judgments. During inference, the model first predicts the most likely family $c^* = \text{argmax}(z)$ and computes $E(x)$; if $E(x) > TH_{c^*}$, the sample is rejected as unknown. We adopt per-family thresholds rather than a single global threshold because the energy-score distributions differ substantially across families. A global threshold would therefore be overly conservative for well-represented families and too permissive for tail families. The coefficient κ governs the trade-off between specificity and sensitivity. A smaller κ admits more samples as known but raises the false-acceptance rate for unknowns, while a larger κ rejects more aggressively at the cost of misclassifying known tail family samples whose energy scores lie further from μ_c . In MEM, we set $\kappa = 1$ in Equation (12), placing the rejection boundary one standard deviation below the class-conditional energy mean to balance these two metrics.

$$TH_c = \mu_c - \kappa \cdot \sigma_c \quad (12)$$

4. Experimental Results

Here are the experimental evaluation and result analysis of the proposed open-set ransomware family identification method. Section 4.1 will describe the experimental dataset and characteristics, model parameter settings, comparison methods, and evaluation indicators. Section 4.2 will present the results of characteristic comparison, overall classification effectiveness, few-shot family identification performance, unknown sample identification performance, and ablation experiments. All experiments were conducted on a Windows 11 Professional (64-bit) system equipped with an AMD Ryzen 5 5600 CPU and an NVIDIA RTX 3060 GPU. The software environment was built on Python 3.8, with PyTorch 2.7.1 and scikit-learn 1.7.2 used for model construction and training.

4.1. Experimental Setup

This section will introduce the dataset used in Section 4.1.1, analyze the model and training parameters in Section 4.1.2, and finally explain the evaluation metrics used in Section 4.1.3.

4.1.1. Data Set and Features

MEM adopts a static analysis perspective, using the Ransomware Combined Structural Feature Dataset, referred to as RCFS, proposed by Moreira et al. [8]. As shown in Table 3, the dataset contains 2675 Windows PE samples covering malicious activity records from 2016 to 2022, including 1408 malicious samples from 40 ransomware families and 1267 Benignware

samples. These characteristics pose two interrelated modeling challenges, as the scarcity of samples in emerging families limits the ability of the model to generalize from the few available examples, while the long-tailed class distribution biases training toward dominant families at the expense of marginal ones. Under the RCFS dataset described, the concatenated feature vector F_i has a fixed dimensionality of 3405 after vocabulary filtering and zero-padding across all six feature categories, including $d_{header} = 52$, $d_{section} = 710$, $d_{entropy} = 143$, $d_{dll} = 500$, $d_{api} = 1000$, and $d_{opcode} = 1000$. The DLL, API, and opcode vocabularies are constructed from each run training set and zero-padded to these fixed maximum lengths to ensure consistent dimensionality across all 30 runs.

Table 3. Number of Collected Portable Executable (PE) Samples in the Dataset.

Family	Sample	Family	Sample	Family	Sample
Benignware	1267	Ryuk	48	BlackBasta	32
Darkside	50	Stop	48	Doppelpaymer	23
Gandcrab	50	Dharma	46	Exorcist	19
Maze	50	Clop	46	Lorenz	16
Netwalker	50	Blackmatter	45	Mountlocker	14
Phobos	50	Babuk	44	NightSky	14
AvosLocker	50	Ragnarok	43	RansomeXX	13
BlackCat	50	PlayCrypt	43	BlackByte	13
Hive	50	Wastedlocker	40	Karma	13
Avaddon	49	Nefilim	39	BianLian	11
Revil	49	Pysa	38	Quantum	6
Zeppelin	49	Makop	35	HolyGhost	4
Conti	48	Thanos	35	Maui	3
Lockbit	48	BlueSky	34		

To address these challenges under a realistic open-set scenario, Benignware is permanently assigned as a known class while 19 ransomware families are randomly selected to form the known-class pool, yielding 20 known and 21 unknown classes per run. The known-class pool is then stratified into training, validation, and test sets at a 70/15/15 ratio to preserve within-family representativeness. The 21 unknown families never appear during any training or meta-training phase, and differ fundamentally from the simulated unknowns in Section 3.2.1, which are temporarily drawn from the known pool during meta-training and therefore not truly unseen. The entire protocol is independently repeated 30 times, each time re-selecting the 19 known families, re-partitioning the data, and re-tuning all baseline hyperparameters on the validation set via grid search so that every compared method shares the identical split within each run. In the few-shot evaluation described in Section 4.2.5, families retaining fewer than 15 training samples after the stratified split are mandatorily included among the 19 known families to ensure that tail-class performance is explicitly measured.

4.1.2. Model Parameter Setup

The open-set ransomware identification architecture proposed in MEM comprises two training phases: MAML and fine-tuning. Both phases optimize the classifier, a set of DNNs responsible for feature representation and multi-class classification of ransomware families, providing the energy scores needed to construct the energy threshold.

This DNN classifier consists of three fully connected hidden layers with 200, 100, and 100 neurons respectively. Each layer uses the Rectified Linear Activation Function (ReLU) and is paired with Dropout at a probability of 0.4 to reduce the risk of overfitting. The input dimension corresponds to the dimension of the static feature vector, and the

output dimension corresponds to the number of known ransomware families covered in the training phase.

In the MAML phase, the meta-learning configuration follows an N-way K-shot protocol as described in Section 3.2. All symbol definitions and their default values are listed in Table 2. The Adam optimizer is used with a weight decay of 0.001, and the batch size is set to 64. The inner loop performs 5 gradient steps per task. Meta-training runs for up to 1000 epochs with an early stopping patience of 3 epochs.

After MAML completes, the model enters the fine-tuning stage as described in Section 3. All loss weights, including λ_k , λ_u , and λ_{ft} , were determined by grid search on the validation set. Qualitatively, setting λ_k too high over-penalizes known-sample energy deviations and degrades classification accuracy, while setting it too low weakens the energy boundary and impairs unknown rejection. Similarly, an excessively large λ_u destabilizes meta-training because the unknown query set is small and noisy, whereas too small a value fails to push unknown energy scores above the threshold. For λ_{ft} , values above 0.5 cause Focal Loss to dominate the fine-tuning objective and underfit head classes, while values near zero reduce the fine-tuning stage to standard cross-entropy, losing the rebalancing effect for tail families.

4.1.3. Evaluation Metrics

To accurately evaluate the model's recognition performance in open-set recognition tasks, MEM selected four evaluation metrics to quantitatively analyze the classification accuracy for known samples, the ability to reject unknown samples, and overall class balance.

Classification Accuracy Cls_{Acc} measures the model's correct classification rate for known family samples, defined as in Equation (13), where $N_{correct}$ is the number of correctly classified known samples in the test set, and N_{sample} is the total number of all known test samples. This metric reflects the ability of the model to remember and discriminate families seen during the training phase and is the most commonly used basic classification performance metric in closed-set recognition scenarios.

$$Cls_{Acc} = \frac{N_{correct}}{N_{sample}} \quad (13)$$

Open-set detection accuracy Det_{Acc} following the detection metric employed by Guo et al. [12], is used to evaluate a model's ability to reject unknown samples. It is defined as shown in Equations (14)–(16). The true positive rate TPR_k is the rate at which the model correctly identifies samples from known ransomware families, and the true negative rate TNR_u is the rate at which it correctly rejects samples from unknown families. It is particularly suitable for dual prediction tasks in open-set scenarios. A higher Det_{Acc} metric indicates that the model is better able to identify ransomware family samples that have never been seen before, effectively reducing the risk of unknown attacks being misclassified as known categories.

$$TPR_k = \frac{TP_k}{TP_k + FN_k} \quad (14)$$

$$TNR_u = \frac{TN_u}{TN_u + FP_u} \quad (15)$$

$$Det_{Acc} = \frac{TPR_k + TNR_u}{2} \quad (16)$$

The macro-averaged F1-score MAF is given by Equation (17), where C is the number of classes including unknown classes, and $F1_i$ is the F1 score for the i -th class. MAF em-

phasizes the average performance across each class and is suitable for evaluating whether a model can account for all families under an imbalanced sample distribution.

$$MAF = \frac{1}{C} \sum_{i=1}^C F1_i \quad (17)$$

The weighted-averaged F1-score *WAF* takes into account the proportion of samples in each class and is defined by the following Equation (18), where N_i is the number of samples in the i -th class and N is the total number of samples in the test set. *WAF* can better reflect the overall predictive performance of the model under the actual data distribution, especially when the proportion of unknown samples is high, it can highlight the true contribution of the unknown identification ability.

$$WAF = \sum_{i=1}^C \frac{N_i}{N} F1_i \quad (18)$$

The Area Under the Receiver Operating Characteristic Curve *AUROC* is a metric that measures a model's intrinsic ability to distinguish between known and unknown samples. This metric evaluates the classifier's overall performance across all possible thresholds. It first requires defining the False Positive Rate FPR_u for unknown samples, as shown in Equation (19). *AUROC* is calculated by plotting the True Positive Rate TPR_k against the False Positive Rate FPR_u and calculating the area under the curve, as shown in Equation (20). This metric is not affected by specific thresholds; a higher value indicates better model performance in distinguishing between known and unknown samples.

$$FPR_u = \frac{FP_u}{FP_u + TN_u} \quad (19)$$

$$AUROC = \int_0^1 TPR_k(FPR_u) dFPR_u \quad (20)$$

4.2. Experimental Results and Analysis

This section evaluates and analyzes the performance of the proposed method under OSR, few-shot learning, and imbalanced data conditions, and verifies the practical value of combining MAML with an energy-assessment design. This section proceeds sequentially as follows: Section 4.2.1 compares method characteristics; Section 4.2.2 analyzes overall classification and unknown detection performance; Section 4.2.3 explores ablation experiments on key components; Section 4.2.4 verifies the stability assessment of open-set recognition; and finally, Section 4.2.5 evaluates the recognition performance of few-shot families.

4.2.1. Characteristic Comparison

To clarify the contribution and positioning of this research method, this section will systematically compare and analyze existing representative methods based on their capabilities and limitations under the three core challenges of ransomware in real-world scenarios: OSR, Few-shot Learning, and Long-tailed Distribution.

Table 4 compares existing methods across the three core challenges, where “O” indicates that a method explicitly addresses the challenge and “X” indicates it does not. Most existing methods target only a single challenge, and their core mechanisms, while solving specific problems, introduce limitations for simultaneously addressing the others. Taking the pioneering OSR research of Bendale and Boulton [23] as an example, OpenMax uses EVT to statistically fit the activation scores of known categories, providing a rejection probability with principled statistical significance for open-set recognition. Therefore, it has been

frequently used as a benchmark comparison method in many recent malware identification studies [11,12] has been applied to handle open classification tasks involving dynamic and static analysis. However, the reliance on statistical fitting becomes a fundamental weakness in ransomware scenarios. The limited sample size and long-tail distribution of the data prevent the collection of enough samples to build a stable tail model, resulting in unreliable rejection boundaries. Furthermore, limitations on feature types also affect its applicability.

Table 4. Comparison of method capabilities under three core challenges of open-set ransomware detection. “O” indicates the method explicitly addresses the challenge; “X” indicates it does not.

Methods	Open-Set Recognition	Few-Shot	Long-Tailed Data Distribution	Feature
OpenMax [23]	O	X	O	Static Analysis
Zhu et al. [10]	X	O	X	Static Analysis
Lu et al. [13]	O	X	X	Dynamic Analysis
CNSNet [12]	O	X	X	Static Analysis
MEM	O	O	O	Static Analysis

Other methods based on static analysis have contributions largely limited to single challenges. Zhu et al. [10] excels at few-shot learning through metric learning, but its closed-set assumption renders it completely incapable of open-set recognition, leading to misclassifications of unknown threats. Moreover, its core architecture does not provide a systematic solution to the long-tail distribution problem. Guo et al. [12] proposed CNSNet, which enhances the rejection decision in open-set recognition with an innovative GAN mechanism, but its design focuses on synthesizing boundary samples, neglecting class balance. Therefore, it offers limited support for addressing long-tail distribution challenges and does not integrate few-shot learning strategies.

Some research has shifted to different feature types. For example, Lu and Wang [13] use dynamic analysis combined with episodic training to simulate open-world scenarios for model training. However, the effectiveness of this simulation depends heavily on the diversity of the training context; otherwise, too few training samples can easily misclassify unknown samples as known families. Furthermore, it suffers from high latency in dynamic analysis itself and is susceptible to interference from anti-sandboxing and evasion techniques. This method also lacks optimization for few-shot and long-tail distribution problems, limiting its applicability in real-world ransomware detection scenarios.

MEM is designed to address the gap identified above. By integrating MAML, an energy-based rejection boundary, and Focal Loss within a static-analysis pipeline, it is the only method in Table 4 that covers all three challenges.

4.2.2. Comparison of Overall Classification Metrics

To verify the practical performance of the architecture proposed in MEM on the open-set ransomware identification task, especially its ability to handle the two core challenges of unknown sample rejection and long-tailed class distribution, this section compares our method with two open-set recognition methods previously applied to malware detection: OpenMax by Bendale and Boulton [23] and CNSNet by Guo et al. [12]. These two baselines were selected because they represent the two principal OSR paradigms available under our experimental constraints. OpenMax employs EVT-based rejection on classifier logits and has served as the standard OSR comparator in recent malware studies [11–13]; CNSNet uses GAN-based unknown synthesis and is the only published method that combines generative rejection with static malware features. Other general-purpose OSR or OOD de-

tectors, such as ODIN and Mahalanobis-distance scoring [24], are designed for deep image classifiers and require architectural components that are incompatible with our shallow DNN and tabular static-feature pipeline, precluding a fair comparison under the same protocol. To further disentangle each component, the ablation study in Section 4.2.3 removes Energy Loss and Center Loss individually, while Section 4.2.4 compares four method combinations that cross two training strategies with two rejection modules, including a Baseline + Energy configuration that shares the same energy-based decision as MEM but replaces MAML with standard DNN training. Following the open-set protocol defined in Section 4.1.1, all experiments are independently repeated 30 times with different random family selections and data splits. We exclude outliers using IQR fences for each metric. The reported result is the mean of the remaining runs.

To eliminate bias caused by the inherent limitations of the classification model, this experiment first observes the performance of the three methods on the Cls_{Acc} metric. As shown in Table 5, all three methods achieved scores above 0.9 on this metric, with our method reaching the highest value of 0.9956, slightly higher than CNSNet [12] 0.9319 and OpenMax [23] 0.9547. This result confirms that all three methods possess the ability to stably identify known families. Therefore, the subsequent differences in unknown sample rejection and overall stability metrics are not due to insufficient basic classifier capabilities, but rather stem from design differences in rejection and class adaptation capabilities.

Table 5. Overall classification and rejection performance on the RCFS dataset under the open-set protocol with 20 known and 21 unknown families. Values are means over 30 independent runs after outlier removal.

Methods	Cls_{Acc}	Det_{Acc}	MAF	WAF
CNSNet [12]	0.9319	0.5354	0.5755	0.2407
OpenMax [23]	0.9547	0.5030	0.4862	0.1639
MEM	0.9956	0.9222	0.7953	0.9369

However, further examination of unknown sample rejection reveals a substantial gap. Based on Det_{Acc} , OpenMax and CNSNet achieved only 0.5030 and 0.5354, respectively, indicating near-random rejection performance. As discussed in Section 4.2.1, OpenMax's EVT-based fitting becomes unreliable when per-family sample sizes are small, while CNSNet's GAN-synthesized unknowns fail to cover the diversity of real ransomware variants. In contrast, MEM achieves a Det_{Acc} of 0.9222 without sacrificing Cls_{Acc} , suggesting that the energy-based rejection boundary remains effective under the evaluated protocol.

To further assess adaptability to class imbalance, we examine MAF and WAF. OpenMax and CNSNet obtain MAF scores of 0.4862 and 0.5755 and WAF scores of 0.1639 and 0.2407, respectively, reflecting poor balance across families of unequal size. MEM achieves MAF of 0.7953 and WAF of 0.9369, indicating improved class-level balance under the long-tailed RCFS distribution. This improvement is attributable to the combination of meta-learned initialization and Focal Loss, which together reduce overfitting to head classes and increase the learning signal from minority and hard-to-classify samples.

4.2.3. Ablation Experiment

To confirm the contribution of Energy Loss and Center Loss, encapsulated in the meta-learning optimization stage of MEM, to the performance of the architecture, we used the integrated architecture described in Section 3 as a baseline and removed the Energy Loss and Center Loss components respectively, analyzing their impact on open-set recognition ability, class correctness, and class balance. The experimental results are shown in Table 6. When the complete architecture simultaneously enables Energy Loss and Center

Loss, Cls_{Acc} reaches 0.9956, Det_{ACC} reaches 0.9222, MAF is 0.7953, and WAF is 0.9369, achieving optimal results across all performance metrics.

Table 6. Ablation study on Energy Loss and Center Loss. Values are means over 30 independent runs after outlier removal.

Energy Loss	Center Loss	Cls_{Acc}	Det_{Acc}	MAF	WAF
O	O	0.9956	0.9222	0.7953	0.9369
X	O	0.9922	0.8883	0.7357	0.8949
O	X	0.9912	0.8793	0.7339	0.8870

When the Energy Loss component is removed while Center Loss is retained, Cls_{Acc} slightly decreases to 0.9922, Det_{ACC} significantly decreases to 0.8883, MAF decreases to 0.7357, and WAF decreases to 0.8949. This result demonstrates that even though the Center Loss does not directly participate in the decision-making process for unknown samples, it promotes the concentration of intra-class features and the discriminative power of inter-class features, forming a clearer feature space and enabling the Energy Loss to more effectively distinguish between known and unknown samples. Conversely, when Center Loss is removed while the Energy Loss is retained, Cls_{Acc} decreases to 0.9912, Det_{ACC} decreases to 0.8793, MAF is 0.7339, and WAF is 0.8870. Although the Center Loss does not directly participate in the rejection decision, its promoted intra-class aggregation and inter-class separation effects result in a better category space, making the feature distribution on which the Energy Function relies more concentrated, thus enhancing the detection efficiency of the rejection logic.

In summary, both Energy Loss and Center Loss contribute meaningfully to the performance of MEM under the evaluated protocol. Removing either component degrades at least one metric. Energy Loss directly governs the unknown rejection decision, while Center Loss improves the intra-class compactness on which the energy boundary relies. These results suggest that the two losses play complementary roles within the tested settings, though their necessity under alternative architectures or thresholding strategies remains to be verified.

4.2.4. Open-Set Recognition Stability Evaluation

To evaluate the stability of the model in an open-set environment, the core objective is to verify whether the model can maintain robust recognition capabilities under dynamic changes within known families. We simulated a real-world scenario where class coverage gradually increases. The experiment fixed the total number of families at 41 and adjusted the number of known ransomware families included in the training data, denoted as N , decreasing sequentially from 40 to 4. We observed the changing trends of the Det_{ACC} and Cls_{ACC} metrics under each setting. During the experiment, only the coverage of families seen during the training phase was adjusted; the remaining unknown families served as test data to verify the stability and generalization ability of each method under open-set conditions. Figures 7–9 plot the average Det_{ACC} , Cls_{ACC} and AUROC, respectively, as the number of known ransomware families N increases from 4 to 40. Error bars in Figures 7–9 show ± 1 standard deviation of the retained values. Four method combinations are compared, formed by crossing two training strategies with two rejection modules: Baseline + Energy, Baseline + OpenMax, MAML + Energy, and MAML + OpenMax. Here Baseline denotes standard DNN end-to-end training, while “MAML” denotes the meta-learning initialization proposed in this study; “Energy” and “OpenMax” refer to the two open-set decision mechanisms evaluated for unknown rejection.

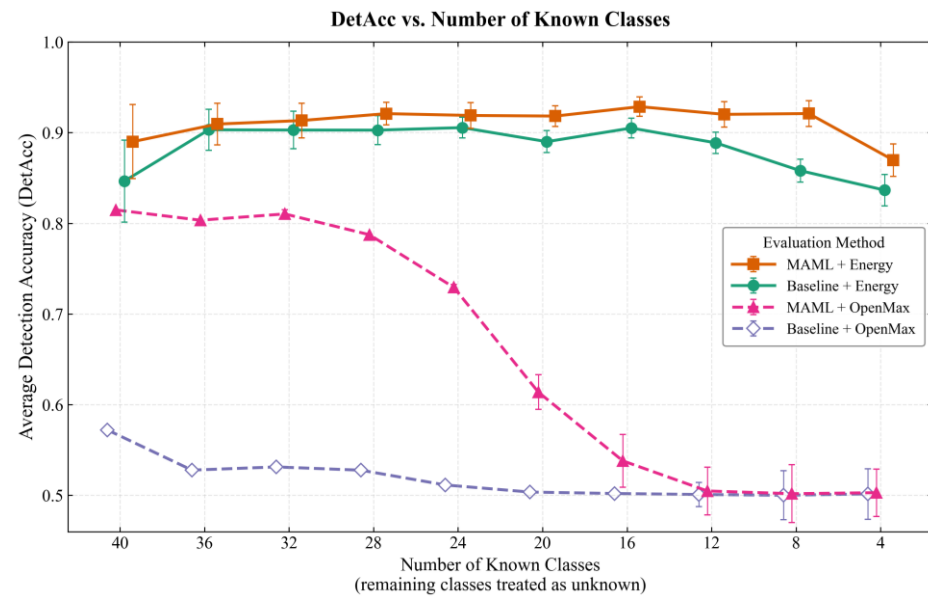


Figure 7. Detection accuracy calculated by Equation (16) as the number of known families N varies from 4 to 40. Values are means over 30 independent runs after outlier removal. Error bars indicate ± 1 standard deviation.

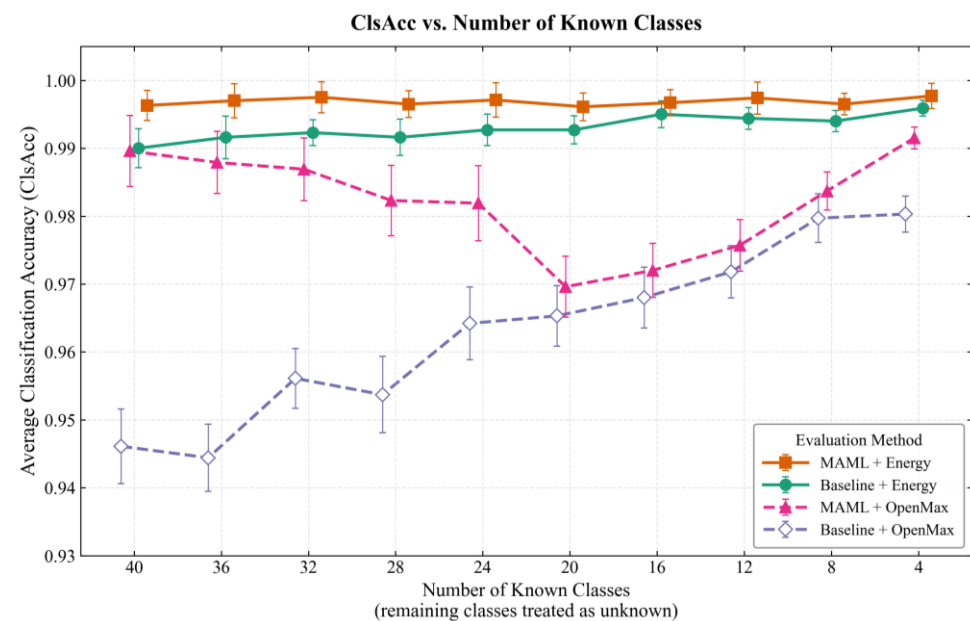


Figure 8. Classification accuracy calculated by Equation (13) across different evaluation methods as the number of known families N varies from 4 to 40. Values are means over 30 independent runs after outlier removal. Error bars indicate ± 1 standard deviation.

To further disentangle the training strategy from the rejection mechanism, Table 7 includes a Baseline + Energy configuration, which shares the same energy-based decision module as MEM but replaces MAML with standard DNN training, thereby isolating the contribution of meta-learning initialization. The Det_{ACC} of Baseline + OpenMax consistently hovers around 0.5, rising only to 0.57 at 40 known families, which is consistent with the EVT fitting limitation discussed in Section 4.2.1. If the number of samples in each family is insufficient, the tail model cannot reliably distinguish unknown samples. The Baseline combined with Energy Function performed relatively stably, but Det_{ACC} remained in the 0.85 to 0.91 range, with an overall unstable fluctuation, reflecting the difficulty of traditional DNNs in consistently constructing energy thresholds with good generalization. In contrast, MAML + Energy maintained a Det_{ACC} between 0.91 and 0.93 across all settings, with no

notable degradation as the number of known families increased. This stability suggests that meta-learned initialization provides the Energy Function with a feature space from which per-family thresholds can be reliably constructed. MAML + OpenMax exhibited an increasing trend, rising after 12 known families and stabilizing near 0.80 above 28, indicating that meta-learning initialization partially mitigates the EVT instability noted in Section 4.2.1.

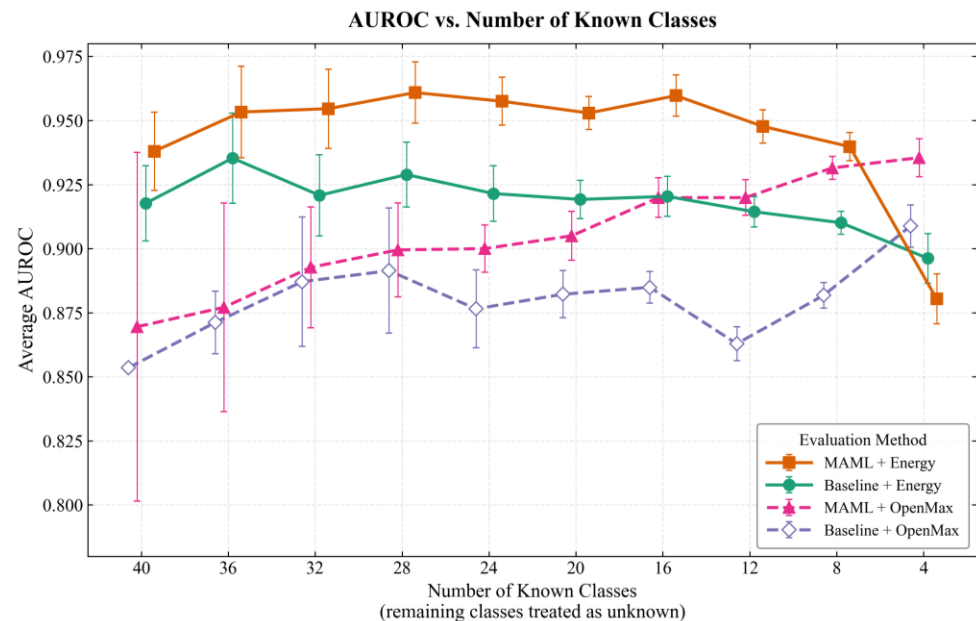


Figure 9. AUROC calculated by Equation (20) across different evaluation methods as the number of known families N varies from 4 to 40. This metric is computed on raw negated energy scores without normalization. Values are means over 30 independent runs after outlier removal. Error bars indicate ± 1 standard deviation.

Table 7. Few-shot family recognition performance. Extreme classification accuracy measures performance strictly for families with fewer than 15 training samples. All these families are mandatorily included as known. Values are means over 30 independent runs.

Training Method	Decision Module	Cls_{Acc}	Det_{Acc}	MAF	WAF	Extreme Cls_{Acc}
Baseline	Energy	0.9637	0.7022	0.1046	0.6890	0.0000
MAML	Energy	0.9917	0.9320	0.7750	0.9419	0.7619
Baseline	OpenMax	0.8093	0.5363	0.1618	0.2061	0.1905
MAML	OpenMax	0.9767	0.6815	0.5675	0.5619	0.8095

To verify whether the aforementioned rejection effectiveness comes at the expense of accuracy for known samples, Figure 8 presents the Cls_{ACC} performance of each method under different numbers of known families. MAML combined with Energy consistently maintains a Cls_{ACC} of 0.995, exhibiting extremely high stability across all stages, indicating that while enhancing rejection capability, it does not significantly negatively impact known samples. The Cls_{ACC} of MAML combined with OpenMax decreases as the number of known families increases from 4 to 20. This is mainly because the increased number of categories with the same number of samples per category leads to crowded classification boundaries, making it easy to misclassify known samples as other families or unknown categories. When the number of families continues to increase beyond 24, the initial features learned by MAML help the model distinguish each family more clearly, reducing misclassifications and gradually improving classification accuracy. Conversely, the Baseline

combined with OpenMax shows a significant downward trend in Cl_{sACC} , continuously dropping from an initial 0.98 to 0.95, indicating that the model cannot stably distinguish between known and unknown samples when the number of categories increases, resulting in a significant imbalance in classification performance and further highlighting the limitations of its rejection strategy in open-set scenarios.

Overall, Figures 7 and 8 indicate that the MAML + Energy combination maintains stable rejection and classification accuracy as the number of known families varies from 4 to 40, while the other three combinations exhibit sensitivity to family coverage.

To further evaluate the inherent ability of distinguishing the difference between known and unknown samples and of going beyond the limitations of a single threshold setting, this section introduces AUROC as an evaluation metric. To ensure a consistent benchmark, the output scores of all methods are converted to a standardized known class confidence score. For the OpenMax method, this score is defined as the overall probability of its known class, i.e., one minus the probability that its output is an unknown class. For the Energy Function, AUROC is computed on the negated energy scores $E(x)$. As shown in Figure 9, the experimental results reveal the differentiated adaptability of different unknown detection mechanisms to changes in task complexity. The MAML combined with the Energy Function proposed in MEM exhibits the most stable performance, with its AUROC value reaching a peak of 0.9601 when the number of known families is 16, and remaining above 0.94 in most cases. In contrast, MAML combined with OpenMax exhibits the opposite trend, with its performance gradually decreasing from a high of 0.9354 when the number of known families is 4 to 0.8694 when the number of known families is 40. The decision performance of the baseline method combined with the energy function fluctuates between 0.8960 and 0.9359, while the decision performance of the baseline combined with OpenMax oscillates significantly between 0.8533 and 0.9097.

The fundamental reason for the opposite performance trends of the two MAML-based methods lies in the different response mechanisms of the Energy Function and OpenMax to task complexity. The performance of MAML combined with the Energy Function improves with the increase in the number of known families, a phenomenon that can be attributed to the normalization effect brought about by task diversity. When the number of known families is small, the task combinations for meta-learning are limited. As the number of families increases, the task combination complexity grows exponentially, forcing MAML to learn a more abstract and generalized feature representation, rather than memorizing subtle differences between specific families. This more robust scope of known concepts allows the Energy Function, as a global indicator, to distinguish more effectively. In contrast, the performance degradation of MAML combined with OpenMax stems from the inherent limitations of its local statistical modeling. OpenMax relies on independent extreme-value theory to fit the activation distribution for each class; this strategy performs well in simplified scenarios with few known families and clear class boundaries. However, as the number of families increases and class boundaries overlap significantly, the activation distributions of each class become chaotic, causing the underlying Weibull distribution to misfit, thus degrading performance. These results indicate that the Energy Function exhibits improved generalization and stability in complex multi-class environments, making it more suitable as a solution for large-scale ransomware classification tasks.

4.2.5. Few-Shot Family Recognition Performance

To verify model performance in real-world few-shot learning scenarios, we focused on evaluating its ability to handle minority classes with fewer than 15 training samples for a single family. This experiment selected all ransomware families with fewer than 15 samples from the RCFS dataset as mandatory families for the training task. Furthermore, to avoid

data bias caused by sample randomness, the experiment was independently repeated 30 times. For each metric, results within the interquartile range were retained and averaged to enhance statistical reliability. To objectively evaluate the ability of the model to identify families with extreme sample counts, the *Extreme Cls_{ACC}* metric was defined, calculating the classification accuracy only for families with fewer than 15 samples in the training set, highlighting model learning and adaptation capabilities for minority classes.

Table 7 shows that Baseline training yields an *Extreme Cls_{ACC}* of 0.0000 for Energy and 0.1905 for OpenMax. These results suggest that standard end-to-end optimization fails to learn discriminative features from families with fewer than 15 samples. In contrast, MAML-based training raises *Extreme Cls_{ACC}* to 0.7619 for Energy and 0.8095 for OpenMax. This improvement is consistent with the rapid-adaptation property discussed in Section 4.2.1.

However, *Extreme Cls_{ACC}* measures only the minority families and does not reveal whether the overall classification is biased toward head classes. The *MAF* and *WAF* metrics provide complementary views for this analysis. *MAF* assesses per-class average performance while *WAF* weights each class by its sample proportion. MAML combined with Energy achieves the highest *MAF* of 0.7750 and the highest *WAF* of 0.9419. These scores indicate balanced performance across families of different sizes. In comparison, MAML combined with OpenMax achieves a competitive *Extreme Cls_{ACC}* of 0.8095 but a lower *WAF* of 0.5619. This gap suggests that its predictions are disproportionately concentrated on minority classes at the expense of overall balance. This contrast indicates that the energy-based boundary and Focal Loss rebalancing distribute prediction quality more evenly across the long-tailed family distribution than the per-class logit calibration of OpenMax.

In summary, the few-shot evaluation indicates that the MAML + Energy + Focal Loss combination improves minority-family recognition while maintaining overall predictive balance under the evaluated protocol. These results complement the stability evaluation in Section 4.2.4 by focusing on performance under extreme data sparsity.

5. Conclusions

As ransomware continues to evolve, new families emerge with very few available samples, making it challenging for detection systems to maintain accurate recognition. Furthermore, the uneven distribution of sample sizes across families leads to biased classification results, where models tend to overfit to families with abundant samples while neglecting those with scarce data. While existing open-set recognition methods have improved detection capabilities, most approaches remain dependent on sufficient training samples to establish reliable rejection boundaries and fail to simultaneously address data imbalance, limited sample availability, and unknown family identification. To mitigate these challenges, we introduce the MEM framework, which integrates MAML, an Energy Function, and Focal Loss to develop a unified open-set recognition system based on static Portable Executable feature analysis. MAML enables the model to rapidly adapt to emerging ransomware families, allowing effective classification even with only a few labeled samples available. Meanwhile, the Energy Function provides a stable rejection mechanism for identifying unknown samples, ensuring that unseen ransomware families are correctly excluded from known classifications. Additionally, Focal Loss is incorporated to reduce the impact of imbalanced data distributions, ensuring that minority families with scarce samples receive adequate recognition during training. While our proposed method demonstrates strong performance in detecting unknown ransomware, several limitations warrant further discussion. Regarding computational overhead, the meta-learning framework involves a nested optimization process, resulting in higher theoretical training complexity

than standard DNN-based models. In terms of scalability, since the task sampling mechanism of MAML primarily depends on the number of classes per task rather than the total number of classes, the impact of increasing the total family count on training overhead is minimal. Although maintaining independent energy thresholds for each known family introduces a marginal storage requirement as the number of families grows, the inference efficiency remains high because the process only requires a single threshold comparison against the predicted class. Lastly, this study currently focuses on static feature analysis for ransomware. Future work will aim to extend this framework to all types of malware detection, thereby enhancing the generalization and versatility of the model in broader malicious software identification scenarios.

Author Contributions: Conceptualization, C.-Y.C.; Methodology, Y.-Y.F.; Validation, C.-Y.C.; Writing—original draft, C.-Y.C.; Writing—review & editing, J.-S.L.; Supervision, Y.-Y.F. and J.-S.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are openly available in Mendeley Data at <https://doi.org/10.17632/yzhcvn7sj5.2>. However, the raw executable datasets presented in this article are not readily available because the dataset contains malware samples that may pose security risks; requests to access the raw datasets should be directed to the corresponding author.

Conflicts of Interest: Author Jung-San Lee was employed by the company Institute for Information Industry. The remaining authors declare that the re-search was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Cyble Research Labs. Cyble Annual Threat Landscape Report 2025. Available online: <https://cyble.com/resources/research-reports/annual-threat-landscape-report-2025/> (accessed on 15 January 2026).
2. Lyda, R.; Hamrock, J. Using Entropy Analysis to Find Encrypted and Packed Malware. *IEEE Secur. Priv.* **2007**, *5*, 40–45. [CrossRef]
3. Ugarte-Pedrero, X.; Balzarotti, D.; Santos, I.; Bringas, P.G. SoK: Deep Packer Inspection: A Longitudinal Study of the Complexity of Run-Time Packers. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy, San Jose, CA, USA, 17 May 2015*; IEEE: Washington, DC, USA, 2015; pp. 659–673.
4. Zhou, B.; Cui, Q.; Wei, X.-S.; Chen, Z.-M. BBN: Bilateral-Branch Network with Cumulative Learning for Long-Tailed Visual Recognition. In *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020*; IEEE: New York, NY, USA, 2020; pp. 9716–9725.
5. Snell, J.; Swersky, K.; Zemel, R. Prototypical Networks for Few-Shot Learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017*; Curran Associates Inc.: Red Hook, NY, USA, 2017; pp. 4077–4087.
6. Yang, J.; Zhou, K.; Li, Y.; Liu, Z. Generalized Out-of-Distribution Detection: A Survey. *Int. J. Comput. Vis.* **2024**, *132*, 5635–5662. [CrossRef]
7. Geng, C.; Huang, S.-J.; Chen, S. Recent Advances in Open Set Recognition: A Survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **2021**, *43*, 3614–3631. [CrossRef] [PubMed]
8. Moreira, C.C.; Moreira, D.C.; Sales, C. A Comprehensive Analysis Combining Structural Features for Detection of New Ransomware Families. *J. Inf. Secur. Appl.* **2024**, *81*, 103716. [CrossRef]
9. Dina, A.S.; Siddique, A.B.; Manivannan, D. A Deep Learning Approach for Intrusion Detection in Internet of Things Using Focal Loss Function. *Internet Things* **2023**, *22*, 100699. [CrossRef]
10. Zhu, J.; Jang-Jaccard, J.; Singh, A.; Welch, I.; AL-Sahaf, H.; Camtepe, S. A Few-Shot Meta-Learning Based Siamese Neural Network Using Entropy Features for Ransomware Classification. *Comput. Secur.* **2022**, *117*, 102691. [CrossRef]
11. Ji, Y.; Zou, K.; Zou, B. Mi-Maml: Classifying Few-Shot Advanced Malware Using Multi-Improved Model-Agnostic Meta-Learning. *Cybersecurity* **2024**, *7*, 72. [CrossRef]

12. Guo, J.; Guo, S.; Ma, S.; Sun, Y.; Xu, Y. Conservative Novelty Synthesizing Network for Malware Recognition in an Open-Set Scenario. *IEEE Trans. Neural Netw. Learn. Syst.* **2023**, *34*, 662–676. [[CrossRef](#)] [[PubMed](#)]
13. Lu, T.; Wang, J. DOMR: Toward Deep Open-World Malware Recognition. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 1455–1468. [[CrossRef](#)]
14. Bai, J.; Wang, J.; Zou, G. A Malware Detection Scheme Based on Mining Format Information. *Sci. World J.* **2014**, *2014*, 260905. [[CrossRef](#)] [[PubMed](#)]
15. Pektaş, A.; Acarman, T. A Dynamic Malware Analyzer against Virtual Machine Aware Malicious Software. *Secur. Commun. Netw.* **2014**, *7*, 2245–2257. [[CrossRef](#)]
16. Santos, I.; Brezo, F.; Ugarte-Pedrero, X.; Bringas, P.G. Opcode Sequences as Representation of Executables for Data-Mining-Based Unknown Malware Detection. *Inf. Sci.* **2013**, *231*, 64–82. [[CrossRef](#)]
17. Wang, Q.; Qian, Q. Malicious Code Classification Based on Opcode Sequences and textCNN Network. *J. Inf. Secur. Appl.* **2022**, *67*, 103151. [[CrossRef](#)]
18. Finn, C.; Abbeel, P.; Levine, S. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning, Sydney, Australia, 6–11 August 2017*; PMLR: Cambridge, MA, USA, 2017; Volume 70, pp. 1126–1135.
19. Wang, H.; Pang, G.; Wang, P.; Zhang, L.; Wei, W.; Zhang, Y. Glocal Energy-Based Learning for Few-Shot Open-Set Recognition. In *Proceedings of the 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023*; IEEE: New York, NY, USA, 2023; pp. 7507–7516.
20. Lin, T.-Y.; Goyal, P.; Girshick, R.; He, K.; Dollar, P. Focal Loss for Dense Object Detection. *IEEE Trans. Pattern Anal. Mach. Intell.* **2020**, *42*, 318–327. [[CrossRef](#)] [[PubMed](#)]
21. Wen, Y.; Zhang, K.; Li, Z.; Qiao, Y. A Discriminative Feature Learning Approach for Deep Face Recognition. In *Proceedings of the Computer Vision—ECCV 2016*; Leibe, B., Matas, J., Sebe, N., Welling, M., Eds.; Springer International Publishing: Cham, Switzerland, 2016; pp. 499–515.
22. Salton, G.; Wong, A.; Yang, C.-S. A Vector Space Model for Automatic Indexing. *Commun. ACM* **1975**, *18*, 613–620. [[CrossRef](#)]
23. Bendale, A.; Boulton, T.E. Towards Open Set Deep Networks. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016*; IEEE: New York, NY, USA, 2016; pp. 1563–1572.
24. Liu, B.; Kang, H.; Li, H.-X.; Hua, G.; Vasconcelos, N. Few-shot Open-set Recognition Using Meta-learning. In *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020*; IEEE: New York, NY, USA, 2020; pp. 8795–8804.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.